01418499 · ภาควิชาวิทยาการคอมพิวเตอร์ มหาวิทยาลัยเกษตรศาสตร์ วิทยาเขตกำแพงแสน

คู่มืออ่านสอบ

AI Troubleshooting Assistant for Everyday Life

เข้าใจระบบทีละบท และอธิบายให้กรรมการฟังได้ด้วยคำของตัวเอง

สอบวันที่ 2 พฤศจิกายน 2569 · ตรวจกับโค้ด branch `express-migration` commit `d31d69f` · จัดทำ 31 ตุลาคม 2569

ทุกตัวเลขนับจากโค้ดหรือ `data/manuals/*.json` ถ้ามาจากเอกสารเดิมจะระบุไว้ ไม่ได้รันระบบหรือแตะฐานข้อมูลตอนจัดทำ

ใช้เล่มนี้อย่างไร

อ่านทีละบท แล้วปิดเล่มอธิบายออกเสียง ถ้าอธิบายไม่ได้ ให้กลับไปอ่านใหม่ เป้าหมายไม่ใช่จำทุกบรรทัด แต่ตอบได้ว่า "ทำงานอย่างไร" และ "ทำไมเลือกแบบนี้"

กล่องในเล่ม	ใช้ทำอะไร
พูดกับกรรมการ	ประโยคตัวอย่าง ปรับเป็นคำของตัวเองได้
ระวัง	สิ่งที่ห้ามพูดเกินจริง หรือจุดที่ระบบยังไม่ทำ
ลองตอบ	คำถามตรวจความเข้าใจ เฉลยอยู่ท้ายเล่ม
ดูในโค้ด	ไฟล์และบรรทัดที่เปิดให้กรรมการดูได้ นับจากรากโปรเจกต์

ลำดับการอ่าน

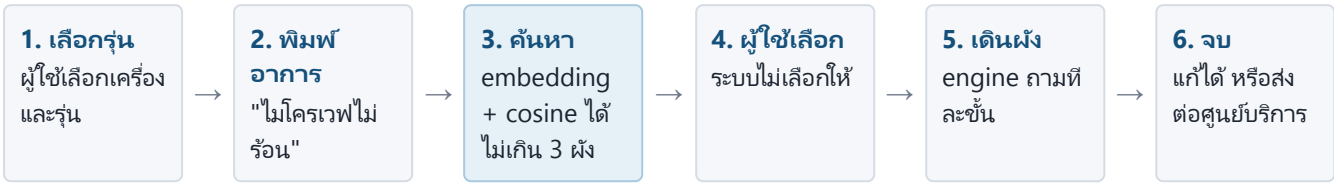
เวลา	อ่าน
วันนี้ (ทำความเข้าใจ)	บท 1–4 ก่อน เป็นแกนของระบบ แล้วตามด้วยบท 5–9
1 พ.ย. (ซ้อมพูด)	บท 10–12, ภาคผนวก, ซ้อมเดโมบนเครื่องตามไฟล์ 17
เช้าวันสอบ	ภาคผนวก ก ตัวเลข และ ภาคผนวก ข สิ่งที่ห้ามพูด

สารบัญ

ภาพรวม	ระบบโดยย่อ และวิธีแนะนำระบบ 1 นาที
บท 1	ผังและ engine: หัวใจของระบบ
บท 2	การค้นหา: embedding และ cosine similarity
บท 3	LLM และความปลอดภัย
บท 4	ความรู้มาจากไหน: 631 ผัง และการร่างจาก PDF
บท 5	Backend: Express และการเดินทางของคำขอ
บท 6	ฐานข้อมูล
บท 7	บัญชีผู้ใช้ สิทธิ์ และ guest
บท 8	Frontend
บท 9	ประวัติ feedback และ dashboard
บท 10	การประเมินผล
บท 11	Deployment และเดโมจากเครื่อง
บท 12	ข้อจำกัด และงานในอนาคต
ภาคผนวก	ตัวเลข · สิ่งที่ห้ามพูด · คำถามยาก · เฉลย · คำศัพท์

ภาพรวมระบบ

ระบบแยก "การหา" ออกจาก "การตอบ" AI ช่วยหาว่าอาการตรงกับหัวข้อไหน แต่คำแนะนำทุกขั้นเป็นข้อความที่เตรียมไว้ในผังก่อนผู้ใช้ถาม



ช่องสีฟ้าคือจุดเดียวบนเส้นทางหลักที่ใช้โมเดล AI และเป็น embedding ที่รันในเครื่อง ไม่ใช่ LLM

ชื่อเรียกของระบบ

Retrieval-guided troubleshooting state machine แปลตรงตัวคือ state machine สำหรับพาแก้ปัญหา ที่ใช้การค้นคืนช่วยเลือกจุดเริ่มต้น ระบบนี้ไม่ใช่ RAG เพราะ RAG ให้ LLM เขียนคำตอบจากข้อความที่ค้นได้ แต่ระบบนี้ LLM ไม่ได้เขียนคำแนะนำ

ประเด็น	ถาม LLM ตรงๆ	RAG	ระบบนี้
คำตอบมาจาก	โมเดลแต่งสด	โมเดลแต่งจากข้อความที่ค้นได้	ผังที่เตรียมไว้ล่วงหน้า
ตรวจก่อนใช้ได้ไหม	ไม่ได้	ตรวจได้แค่ข้อความต้นทาง	ตรวจผังได้ก่อนเผยแพร่
ขั้นอันตราย	ขึ้นกับ prompt	ขึ้นกับ prompt	บังคับด้วยโค้ดฝั่งเซิร์ฟเวอร์
เมื่อไม่รู้	มักตอบอยู่ดี	ขึ้นกับ prompt	บอกว่าไม่พบ หรือส่งต่อศูนย์บริการ
ข้อแลก	ยืดหยุ่นกว่า ตอบได้หลายเรื่อง		ตอบได้เฉพาะอาการที่มีผัง (631 ผัง)

ส่วนประกอบ

ส่วน	เทคโนโลยี	หน้าที่
Frontend	React 19, Vite, TanStack Query, Tailwind	หน้าจอผู้ใช้และ admin
Backend	Express 4, TypeScript, mysql2	API, engine, ค้นหา, ตรวจสิทธิ์
ฐานข้อมูล	MySQL (utf8mb4)	ผัง, session, ผู้ใช้, feedback, ร่าง
Embedding	MiniLM หลายภาษา (ONNX) 384 มิติ	แปลงข้อความเป็นเวกเตอร์ในเครื่อง
LLM (เสริม)	Groq API	เรียบเรียงถ้อยคำ และช่วย admin ร่างผัง

พูดกับกรรมการ: แนะนำระบบ 1 นาที

"เวลาเครื่องใช้ไฟฟ้ามีปัญหา คนส่วนใหญ่ไม่เปิดคู่มือ หรือถาม AI แชตซึ่งอาจแต่งขั้นตอนที่คู่มือไม่ได้พูด ระบบของผมแปลงส่วนแก้ปัญหาของคู่มือ Samsung 21 เล่มเป็นผังขั้นตอน 631 ผัง ผู้ใช้เลือกรุ่น พิมพ์อาการเป็นภาษาพูด ระบบใช้ embedding หาอาการที่ใกล้ที่สุดไม่เกิน 3 อันให้เลือกเอง จากนั้นพาตรวจทีละขั้นตามผัง ขั้นอันตรายต้องยืนยันคำเตือนก่อน และถ้าคู่มือไม่บอกทางแก้ ระบบจะส่งต่อศูนย์บริการแทนการเดา LLM ไม่ได้เป็นคนคิดคำแนะนำ"

ผังและ engine: หัวใจของระบบ

ผังเหมือนแบบสอบถามที่บอกว่า "ตอบใช่ ไปข้อนี้ ตอบไม่ ไปข้อนั้น" engine คือโปรแกรมที่เดินตามกติกานั้น โดยไม่ใช้ AI

โหนด 5 ชนิด

ชนิด	หน้าที่	action ที่รับ	ใน 631 ผัง
`checkpoint`	ถาม ใช่/ไม่ใช่	`answer` แล้วไป `on_yes` หรือ `on_no`	2,146
`instruction`	สั่งให้ทำ 1 อย่าง	`continue` หรือ `confirm_safety` ถ้าเป็นขั้นอันตราย	1,107
`resolution`	จบ: แก้ได้ หรือเป็นอาการปกติ	ไม่รับแล้ว	706
`escalation`	จบ: ส่งต่อศูนย์บริการ	ไม่รับแล้ว	780
`input`	ให้พิมพ์ค่า เช่น รหัส error	`input` ตรวจด้วย regex	0

> **ระวัง**
> โค้ดรองรับ `input` แต่ข้อมูล 631 ผังไม่มีโหนดชนิดนี้เลย จึงอย่าบอกว่าเดโมผังเดิมแล้วจะเห็นขั้นกรอกรหัส error

ตัวอย่างผังจริง

ผัง `samsung_ge83mdt_um_not_cooked_at_all` "อาหารไม่สุกเลย" แถวสีเขียวคือเส้นทางที่เดโม

โหนด	ชนิด	ข้อความ (ย่อ)	ใช่ / ต่อไป	ไม่ใช่
`n1`	checkpoint	ตั้งเวลาถูกต้องและกด Start แล้วใช่หรือไม่?	`n2`	**`n1f`**
`n1f`	instruction ⚠	ตั้งเวลาให้ถูกต้องและกดปุ่ม Start	**`n1v`**	
`n1v`	checkpoint	หลังทำตามแล้ว อาการหายหรือไม่?	**`n_resolved`**	`n2`
`n2`	checkpoint	ประตูเตาปิดสนิทแล้วใช่หรือไม่?	`n3`	`n2f`
`n2f`	instruction ⚠	ปิดประตูให้สนิทแล้วเริ่มปรุงอีกครั้ง (คู่มือหน้า 37)	`n2v`	
`n2v`	checkpoint	หลังทำตามแล้ว อาการหายหรือไม่?	`n_resolved`	`n3`
`n3`	checkpoint	ใช้ไฟเกินกำลังวงจรจนฟิวส์ขาดหรือเบรกเกอร์ตัดใช่หรือไม่?	`n3e`	`n_escalate`
`n_resolved`	resolution	อาหารสุกตามปกติแล้ว (`user_fixed`)	จบ	
`n3e`	escalation	คู่มือให้ตรวจเรื่องนี้แต่ไม่บอกวิธีแก้ ส่งต่อศูนย์ (`handoff_informed`)	จบ	
`n_escalate`	escalation	ไม่พบสาเหตุในรายการตรวจ ส่งต่อศูนย์ (`handoff_unknown`)	จบ	

engine ทำงานอย่างไร

ฟังก์ชันหลักคือ `submitAction` ทำ 5 อย่าง: ปฏิเสธถ้า session จบแล้ว, หาโหนดปัจจุบัน, เรียก `resolveNextNode` เพื่อตัดสินโหนดถัดไป, เปลี่ยนสถานะเป็น `completed` ถ้าถึงโหนดจบ, คืน session ใหม่พร้อมโหนดที่จะแสดง

การตัดสินใจของ checkpoint มีบรรทัดเดียว และไม่มี AI:

```
nextNodeId: action.value === 'yes' ? node.on_yes : node.on_no,
```

คำศัพท์ 2 คำที่ต้องอธิบายได้:

- **Deterministic:** ผังเดิมและคำตอบเดิม ได้ผลเดิมทุกครั้ง
- **Pure function:** engine รับค่าเข้าแล้วคืนค่าออก ไม่เรียกฐานข้อมูลหรือเน็ตเอง การบันทึกเป็นหน้าที่ของ `TraversalService` จึงทดสอบง่าย และใช้เล่นประวัติซ้ำได้ (บท 9)

ด่านความปลอดภัย

```
if (node.safety_critical) {
  if (action.type === 'continue') {
    throw new SafetyConfirmationRequiredError(node.node_id);   // ตอบ HTTP 400
  }
  ...
}
```

มี 368 โหนดที่เป็น `safety_critical` ปุ่มที่กดไม่ได้บนหน้าเว็บเป็นแค่ด่านแรก ถ้ายิง API ตรงด้วย `continue` เซิร์ฟเวอร์ก็ยังปฏิเสธ เพราะกฎอยู่ใน engine

> **พูดกับกรรมการ**
> "การตัดสินเส้นทางไม่ได้ใช้ AI ครับ ผังกำหนดไว้ล่วงหน้า engine ดูแค่โหนดปัจจุบันกับคำตอบ แล้วไปโหนดถัดไปตามผัง ถ้าเป็นขั้นอันตราย ต้องส่งการยืนยันเท่านั้น ถึงยิง API ข้ามหน้าเว็บ เซิร์ฟเวอร์ก็ปฏิเสธ"

> **ระวัง**
> การยืนยันคือการติ๊ก ระบบรู้ไม่ได้ว่าผู้ใช้ถอดปลั๊กจริงหรือไม่ และด่านนี้ใช้กับโหนด instruction เท่านั้น

> **ลองตอบ**
> 1. ผู้ใช้ 2 คนเลือกผังเดียวกันและตอบเหมือนกัน จะได้ขั้นตอนเหมือนกันไหม เพราะอะไร
> 2. ยิง `continue` ไปที่โหนด safety_critical แล้วเกิดอะไรขึ้น ทำไมล็อกแค่ปุ่มบนหน้าเว็บไม่พอ
> 3. ทำไมปิด LLM แล้วยังเดินผังได้

ดูในโค้ด `backend/src/traversal-engine/traversal-engine.ts` : `submitAction` 41-72, checkpoint 85, ด่านความปลอดภัย 90-97 · ข้อมูล `data/manuals/samsung_ge83mdt_um_de68_04111e.json`

การค้นหา: embedding และ cosine similarity

ส่วนค้นหาเหมือนบรรณารักษ์ที่ชี้ว่าควรเปิดบทไหน ชี้ผิดได้ จึงเสนอหลายตัวเลือกให้ผู้ใช้เลือกเอง

สิ่งที่ถูกค้นคืออะไร

ระบบไม่ได้ค้น chunk ของคู่มือ แต่ค้น "ป้ายชื่ออาการ" ของผัง แต่ละผังมีป้ายหลายแบบ คือ ชื่ออาการไทย ชื่ออาการอังกฤษ และคำพ้อง (`entry_symptom_aliases`) รวมทั้งระบบมี **3,814 ป้าย** ซ้ำกันบางส่วน เหลือข้อความไม่ซ้ำ **1,936 ข้อความ** ซึ่ง embed ครั้งเดียวแล้วใช้เวกเตอร์ร่วมกัน

ขั้นตอนการค้นหา

ตรวจคำค้น
ไม่ว่าง ไม่เกิน 200 ตัวอักษร
→
แปลงเป็นเวกเตอร์
384 มิติ
→
กรองรุ่น
เหลือเฉพาะป้ายของ `manualId`
→
Cosine
เก็บคะแนนสูงสุดต่อผัง
→
ตัด
≥ 0.55 และไม่เกิน 3

Embedding คืออะไร

Embedding แปลงข้อความเป็นเลข 384 ตัว ข้อความที่ความหมายใกล้กันจะได้เวกเตอร์ที่ชี้ไปทางใกล้กัน "ไม่ร้อน" กับ "food is not heated" จึงเทียบกันได้แม้คนละภาษา โดยไม่ต้องตัดคำไทยเอง

โมเดลคือ `Xenova/paraphrase-multilingual-MiniLM-L12-v2` แบบ quantized (q8) ไฟล์ประมาณ 56 MB เก็บใน repo และตั้ง `allowRemoteModels = false` จึงไม่โหลดจากเน็ตตอนรัน

Cosine similarity

$$\cos(q, d) = \frac{q \cdot d}{\|q\| \times \|d\|}$$

วัดมุมระหว่างเวกเตอร์ ชี้ทางเดียวกันได้ 1 ตั้งฉากกันได้ 0 ไม่สนใจความยาวของเวกเตอร์

คำนวณด้วยมือ (เวกเตอร์ 3 มิติ)

สมมติคำค้น q = (1, 2, 2) ดังนั้น ‖q‖ = √(1+4+4) = 3

ผัง	d	q · d	‖d‖	cos	เกณฑ์ 0.55
A	(2, 1, 2)	2+2+4 = 8	3	8/9 = 0.889	ผ่าน
B	(0, 0, 1)	0+0+2 = 2	1	2/3 = 0.667	ผ่าน
C	(2, −1, 0)	2−2+0 = 0	√5	0	ตก

ถ้าผังหนึ่งมีหลายป้าย ระบบใช้**คะแนนสูงสุด**ของป้ายในผังนั้น ถ้าคะแนนเท่ากันจะเรียงตาม `graphId` เพื่อให้ผลคงที่

ตัวอย่างผลจริง และเหตุผลที่เสนอ 3 ผัง

คำค้น (รุ่น GE83MDT)	ผล
"ไมโครเวฟไม่ร้อน"	`not_start` 0.845 อันดับ 1 · `not_cooked_at_all` 0.822 อันดับ 2

"วันนี้ฝนตก"	0.301 ไม่ถึงเกณฑ์ ระบบตอบว่าไม่พบ

ค่าจาก `docs/STUDY_GUIDE.md:604-629` ซึ่งวัดด้วย `backend/scripts/study-measure.js` ไม่ได้รันใหม่ตอนจัดทำเล่มนี้

อาการที่ถูกอยู่อันดับ 2 นี่คือเหตุผลที่ระบบไม่เลือกให้เอง

การตัดสินใจที่ต้องตอบได้

คำถาม	คำตอบ
ทำไมต้องเลือกรุ่นก่อน	ถ้าไม่กรอง "ไมโครเวฟไม่ร้อน" มี 120 ผังผ่านเกณฑ์ และ 3 อันดับแรกได้ 1.000 เท่ากัน เพราะมีคำฟ้องตรงตัวในหลายรุ่น
ทำไมไม่ใช้ vector DB	ข้อความไม่ซ้ำแค่ประมาณ 2,000 ข้อความ เวกเตอร์ใช้ RAM ราว 3 MB ไล่เทียบทุกตัวเร็วพอ และได้ผล exact การเพิ่มฐานข้อมูลอีกตัวเพิ่มความซับซ้อนโดยไม่จำเป็น
ทำไมไม่เก็บเวกเตอร์ใน DB	คำนวณตอนบูตแล้วเก็บใน RAM ไม่มีปัญหาเวกเตอร์ค้างเมื่อผังถูกแก้ ข้อแลกคือบูตช้าลงราว 1 นาที แต่ไม่บล็อกการเปิดเซิร์ฟเวอร์ ระหว่างนั้นค้นหาได้ 503
ทำไม embedding ในเครื่อง	ไม่มีค่าใช้จ่ายต่อคำค้น ข้อความผู้ใช้ไม่ออกนอกเซิร์ฟเวอร์ ไม่พึ่งเน็ตหรือ rate limit ข้อเสียคือใช้ RAM ราว 250–300 MB และโมเดลเล็กอาจคุณภาพต่ำกว่า
ความซับซ้อน	O(N·d) ต่อคำค้น เมื่อ N คือจำนวนป้าย d = 384 กรองรุ่นแล้วเฉลี่ยราว 180 ป้าย
ถ้าข้อมูลโตขึ้น	ใช้ ANN เช่น HNSW หรือ vector DB, cache เวกเตอร์, hybrid search กับ BM25

ระวัง

- คะแนนคือความคล้ายของข้อความ **ไม่ใช่ความน่าจะเป็นที่วินิจฉัยถูก** 0.8 ไม่ได้แปลว่าถูก 80% และ engine ไม่ได้ใช้คะแนนเลือกเส้นทาง
- **0.55 ไม่ได้ปรับกับข้อมูลปัจจุบัน** มาจากชุด 56 ข้อที่ AI ร่าง ของคู่มือแอร์ในโปรเจกต์เก่า
- ภายในรุ่นเดียวกัน คะแนนสูงกันหมด เพราะคำว่า "ไมโครเวฟ" ดันคะแนน ("ไมโครเวฟไม่ร้อน" ผ่าน 8 จาก 9 ผัง) เกณฑ์จึงตัดได้แค่ของที่ไม่เกี่ยวเลย

พูดกับกรรมการ

"ผมแปลงคำค้นและซื่ออาการเป็นเวกเตอร์ด้วยโมเดลในเครื่อง กรองเฉพาะรุ่นที่เลือก แล้วเทียบด้วย cosine ถ้าไม่มีผังไหนถึง 0.55 ระบบตอบว่าไม่พบ และไม่เดา แต่ค่านี้ยังไม่ได้ปรับกับข้อมูลชุดปัจจุบัน ผมจึงไม่ได้อ้างว่าเป็นค่าที่ดีที่สุด"

ลองตอบ

1. q = (1, 0, 0) กับ d = (1, 1, 0) ได้ cosine เท่าไร ผ่าน 0.55 ไหม
2. กรรมการถามว่า "คะแนน 0.82 แปลว่าระบบมั่นใจ 82% ใช่ไหม" จะตอบว่าอย่างไร
3. ทำไมมี 3,814 ป้าย แต่ embed แค่ 1,936 ครั้ง

ดูในโค้ด `backend/src/symptom-search/symptom-search.service.ts` : ค่าคงที่ 11-12, ค้นหา 160-191 · `symptom-matcher.ts` : cosine 20-41, ป้าย 43-62, จัดอันดับ 64-88, ตัด 90-95 · `backend/src/embedding/embedding.service.ts:34, 46-55`

LLM และความปลอดภัย

LLM อยู่ 2 จุด ทั้งสองจุดไม่ได้ตัดสินเส้นทาง และถ้าปิดไป ผู้ใช้ยังค้นหาและเดินผังได้ครบ

	1. เรียบเรียงถ้อยคำ (rephrase)	2. ร่างผังจาก PDF (authoring)
ใครเห็นผล	ผู้ใช้ พร้อมลิงก์ดูข้อความต้นฉบับ	admin เท่านั้น จนกว่าจะกด publish
เปิดอย่างไร	ปิดเป็นค่าเริ่มต้น เปิดด้วย `REPHRASE_ENABLED=true`	ต้องตั้ง `AUTHORING_LLM_MODEL` และ key
ส่งอะไรให้ LLM	ข้อความของโหนดเท่านั้น ไม่ส่งคำค้นของผู้ใช้	ข้อความ PDF ทีละ 2 หน้า
ตั้งค่า	temperature 0, timeout 3 วินาที	temperature 0, JSON mode, timeout 60 วินาที, max_tokens 6000
ถ้าล้ม	ใช้ข้อความต้นฉบับ	ร่างมีสถานะ `failed`

ด่านตรวจ rephrase

ไม่ rephrase โหนดอันตรายและโหนดจบเลย ส่วนโหนดอื่น ผลของ LLM ต้องผ่าน `checkRephrase` ทุกข้อ ไม่อย่างนั้นทิ้งแล้วใช้ต้นฉบับ:

- ไม่ว่าง ไม่มีลิงก์ ไม่มี markdown หรือ HTML ที่ต้นฉบับไม่มี และไม่แตกหลายย่อหน้า
- ความยาวอยู่ระหว่าง 0.5 ถึง 1.6 เท่าของต้นฉบับ
- ชุดตัวเลขต้องเหมือนเดิมทุกตัว โดยแปลงเลขไทยก่อน
- คำภาษาอังกฤษ เช่น รุ่น รหัส ชื่อปุ่ม ต้องครบและไม่มีคำเกิน
- จำนวนคำว่า "ไม่" ต้องเท่าเดิม ไม่นับ "หรือไม่" เพื่อกันการกลับความหมาย

> **ระวัง**
> ด่านนี้ตรวจรูปผิว ไม่ได้เข้าใจความหมาย ถ้า LLM เปลี่ยนคำกริยาแต่ตัวเลขและคำปฏิเสธเท่าเดิม ก็อาจผ่านได้ จึงปิดเป็นค่าเริ่มต้นและไม่ใช้กับขั้นอันตราย

กันการแต่งข้อมูล (hallucination) หลายชั้น

1. คำแนะนำไม่ได้สร้างตอนผู้ใช้ถาม จึงไม่มีการแต่งสดบนเส้นทางหลัก
2. การอ้างอิงมาจากฟิลด์ของผัง (`source` , `page_range`) LLM ไม่ได้พิมพ์ตอนรัน
3. rephrase มีด่านตรวจ และแสดงข้อความต้นฉบับให้กดดูได้
4. ร่างจาก LLM ต้องผ่าน validator และคนต้องกด publish เอง

> **ระวัง: อย่าพูดว่า "ระบบไม่มีวัน hallucinate"**
> ความเสี่ยงย้ายไปอยู่ขั้นเตรียมข้อมูล ตอนทำ chunk LLM เคยแต่งคำว่า "blown/tripped" ที่ PDF ไม่มี สคริปต์ตรวจจับได้ แต่ความหมายและคำแปลยังไม่มีคนตรวจ (บท 4)

ความปลอดภัยหลายชั้น

ชั้น	กลไก
ข้อมูล	368 โหนด `safety_critical` พร้อม `safety_warning`
Engine	รับเฉพาะ `confirm_safety` ถ้าส่ง `continue` ได้ 400 `SAFETY_CONFIRMATION_REQUIRED`
หน้าจอ	`SafetyGateStep` กล่องแดง ปุ่มกดไม่ได้จนกว่าจะติ๊ก
LLM	ไม่ rephrase ขั้นอันตราย ทั้งฝั่งเซิร์ฟเวอร์และหน้าจอ
Prompt ร่างผัง	สั่งห้ามตัดหรือลดคำเตือน และห้ามเพิ่มข้อมูลจากความรู้ของโมเดล
Validator	`E_SAFETY_NO_WARNING` และ regex คำเสี่ยงไทย-อังกฤษ เช่น ปลั๊ก ไฟฟ้า แก๊ส ถ้าพบแต่ไม่ตั้งด่านเตือนด้วย `W_NODE_RISK`

เมื่อข้อมูลไม่พอ

สถานการณ์	ระบบทำอะไร
คำค้นไม่ถึง 0.55	"ไม่พบ... ระบบไม่เดาอาการให้" พร้อมลิงก์อาการทั้งหมด
ตอบไม่ใช่จนหมดรายการตรวจ	escalation `handoff_unknown`
คู่มือบอกสาเหตุ แต่ไม่บอกวิธีแก้	escalation `handoff_informed`
ระบบค้นหายังโหลดไม่เสร็จ	503 หน้าจอแจ้งเป็นข้อมูล และใช้หน้าอาการทั้งหมดได้
LLM ล่ม	ใช้ข้อความต้นฉบับ ผู้ใช้เดินผังได้ตามปกติ

> **พูดกับกรรมการ**
> "ผมไม่ให้ LLM อยู่บนเส้นทางที่สร้างคำแนะนำ เพราะงานนี้เกี่ยวกับไฟฟ้าและความร้อน การใช้ prompt สั่งเป็นได้แค่การขอร้อง แต่กฎในโค้ดเป็นการบังคับ LLM จึงทำแค่ปรับภาษา ซึ่งปิดอยู่เป็นค่าเริ่มต้น และช่วยร่างผังให้คนตรวจ"

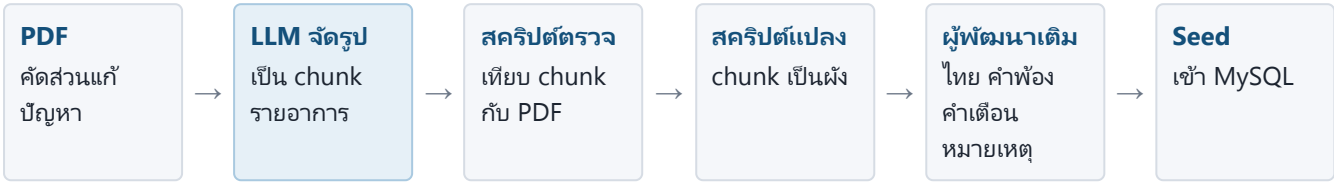
> **ลองตอบ**
> 1. ระบบนี้เป็น RAG ไหม ต่างกันตรงไหน
> 2. ถ้า LLM เปลี่ยน "ถอดปลั๊ก 30 นาที" เป็น "ถอดปลั๊ก 60 นาที" ด่านใดจับได้
> 3. Groq ล่มระหว่างเดโม อะไรยังใช้ได้ อะไรใช้ไม่ได้

ดูในโค้ด `backend/src/rephrase/rephrase.service.ts:40, 56-58` · `rephrase-guard.ts:26-61` · `groq-llm.client.ts:5-11` · `backend/src/authoring/draft-llm.client.ts:33-62` · `frontend/src/components/step/SafetyGateStep.tsx`

ความรู้มาจากไหน: 631 ผัง และการร่างจาก PDF

ผังมาได้ 2 ทาง ทาง A เป็นที่มาของ 631 ผังทั้งหมด ทาง B เป็นเครื่องมือของ admin สำหรับเพิ่มคู่มือใหม่

ทาง A: ทำนอกระบบ (ที่มาของ 631 ผัง)



ทำไมแบ่งตามอาการ: ส่วนแก้ปัญหาของคู่มือเป็นตาราง "อาการ สาเหตุ วิธีแก้" อยู่แล้ว การตัดทีละ 500 ตัวอักษรจะผ่าอาการกลางคัน จึงตัดหนึ่งอาการต่อหนึ่งชิ้น ไม่มี chunk size หรือ overlap ในระบบ

ผ่านการตรวจแค่ไหน

ตรวจอะไร	ผล
ผังกับ PDF ด้วยสคริปต์ (16 ต.ค.)	จาก 644 ผังในรายงาน (รวมผังแอร์ 13 ผังที่ไม่อยู่ในระบบแล้ว): ผ่าน 559, ข้อยกเว้นที่ตรวจแล้ว 6, WARN 18, FAIL 0, ตรวจไม่ได้ 61 (ตัดผังแอร์แล้วเหลือ 48)
โครงสร้าง	`validate_graph.py` และ `simulate_paths.py` ตรวจว่าไปถึงทุกโหนด ไม่มีทางตันและด่านความปลอดภัยครบ
คนตรวจคำแปลและลำดับขั้น	**ยังไม่ได้ทำอย่างเป็นระบบ** (เอกสารเขียนไว้เองว่ายังไม่ได้ทำ)

> **ระวัง**
>
> สคริปต์ที่ให้ผลตรวจ PDF ชุดนี้ถูกลบจาก repo แล้ว (กู้ได้จาก commit `76c0911`) จึงรันโชว์ตอนนี้ไม่ได้ทันที และ "FAIL 0" วัดว่าคำและตัวเลขอยู่ในหน้าที่อ้าง ไม่ได้วัดความหมาย

author_note: ข้อความไหนผู้พัฒนาเขียนเอง

`author_note` เป็นหมายเหตุบอกที่มา ไม่ใช่ข้อความที่ผู้ใช้เห็น มี 1,264 โหนด และเก็บในฐานข้อมูล แต่ backend ไม่ได้ SELECT ขึ้นมา ผู้ใช้จึงไม่เห็น

กลุ่ม (จัดด้วยกฎคำสำคัญ ตัวเลขเป็นค่าประมาณ)	โหนด
คำเชื่อม "หลังทำตามแล้ว อาการหายหรือไม่?" (นับตรงตัว)	720
ประกอบจากหลายหน้าในเล่มเดียวกัน หรือโยงชื่อปุ่ม/คำศัพท์	~313
ผู้พัฒนาเพิ่มขั้นหรือคำเตือน โดยดึงจากหน้าอื่นในเล่ม	~57
มีส่วนที่ผู้พัฒนาเขียนหรือสรุปเอง: คำเตือนทั้งก้อน 36 โหนด (ทีวี) และบางส่วนอีกราว 45 โหนด	~81
บันทึกว่าคู่มือไม่บอกวิธี ผู้พัฒนาจึงไม่เติมรายละเอียด	~93

นอกจากนี้ 64 โหนดมีหมายเหตุว่า "ควรให้ผู้ตรวจยืนยัน" ซึ่งยังไม่ได้ตรวจ

ระวัง: อย่าพูดว่า

"81 โหนดไม่ได้มาจากคู่มือเลย" หรือ "คำเตือนที่เพิ่มเข้มกว่าคู่มือทุกข้อ" เพราะยังไม่ได้เทียบทีละข้อ และยังไม่มีผู้เชี่ยวชาญตรวจ

ทาง B: admin ร่างผังจาก PDF

อัปโหลด	→	แยกข้อความ	→	LLM ร่าง	→	Validator	→	Admin ตรวจ	→	Publish
≤ 10 MB เลือก ≤ 10 หน้า		unpdf ไม่ทำ OCR		ทีละ 2 หน้า ตอบ JSON		24 กฎ error, 5 กฎ warning		แก้ JSON ได้		transaction เดียว

- **สถานะร่าง:** `drafting` → `ready` หรือ `failed` → `published` หรือ `rejected`
- **ตัวอย่างกฎ error:** `E_UNREACHABLE` (มีโหนดที่ไปไม่ถึง), `E_NO_EXIT` (ไม่มีทางจบ), `E_SOURCE_PAGE_OUTSIDE` (อ้างหน้าที่ไม่ได้อัปโหลด)
- **Publish:** มี error ≥ 1 ถูกปฏิเสธ 409 ถ้ามี warning ต้องกดยอมรับ และเพิ่มได้เฉพาะคู่มือใหม่ ไม่เขียนทับของเดิม
- **ใช้ได้ทันที:** `reload()` สร้างข้อมูลชุดใหม่ให้เสร็จก่อนแล้วค่อยสลับ และ `addGraphs()` embed เฉพาะข้อความใหม่ จึงไม่ต้องรีสตาร์ต

ระวัง

- ตอน publish ระบบทิ้ง `source_page` รายโหนด เหลือช่วงหน้าของผัง
- อาการที่คร่อมระหว่างกลุ่ม 2 หน้าอาจหาย เพราะสั่งให้ข้ามอาการที่ถูกตัดขอบ
- เอกสารบันทึกว่า flow นี้ยังไม่ได้ทดสอบกับ PDF และ Groq จริงหลังย้ายมา Express ถ้าจะเดโม ต้องลองก่อน

พูดกับกรรมการ: 631 ผังใครทำ

"ทั้งหมดมาจาก pipeline นอกระบบครับ ใช้ LLM จัดรูปข้อความจาก PDF เป็นรายอาการ แปลงเป็นผังด้วยสคริปต์ แล้วผมเติมภาษาไทยกับคำเตือน ตรวจอัตโนมัติกับ PDF แล้วไม่พบ FAIL แต่มี 48 ผังที่ตรวจอัตโนมัติไม่ได้ และคำแปลกับลำดับขั้นยังไม่มีคนตรวจอย่างเป็นระบบ ผมบอกข้อนี้ตรงๆ เพราะเป็นความเสี่ยงหลักของข้อมูล"

ลองตอบ

1. 631 ผังมาจากการอัปโหลด PDF ในเว็บใช่ไหม
2. ทำไมไม่ตัด chunk ทีละ 500 ตัวอักษร
3. ถ้า LLM ร่างผังที่มีโหนดไปไม่ถึง จะเกิดอะไรขึ้นเมื่อกด publish

---

ดูในโค้ด `scripts/convert_chunks_to_graphs.py` · `backend/scripts/seed-graphs.js` · `backend/src/authoring/drafts.service.ts:85-117, 183-213` · `graph-validator.ts:85-421` · `draft-manual.ts:99-120` · `docs/ops/VERIFY-GRAPHS-AGAINST-SOURCE.md:15-32`

Backend: Express และการเดินทางของคำขอ

ไฟล์ `app.ts` เป็นแผงประกอบ สร้างทุกชิ้นด้วย `new` แล้วผูก URL กับแต่ละโมดูล อ่านจากบนลงล่างก็รู้ว่าใครใช้ใคร

ชั้นของโค้ด

Route รับคำขอ → **DTO** ตรวจข้อมูล → **Service** ตรรกะของงาน → **Store / Repository** SQL → **MySQL**

ชั้น	ไฟล์	ถ้ากรรมการถามเทียบกับ NestJS
Route	`*.routes.ts`	Controller
DTO	`*.dto.ts`	DTO + ValidationPipe: ตรวจเองทีละช่อง และคืนเฉพาะช่องที่ประกาศ
Service	`*.service.ts`	Provider
Middleware สิทธิ์	`auth.middleware.ts`	Guard
Error handler	`error-handler.ts`	Exception filter

ลำดับตอนบูต

ตรวจ `JWT_SECRET` → ต่อ MySQL → โหลดผังเข้า RAM → โหลดอุปกรณ์ → เริ่มโหลดโมเดลค้นหาแบบไม่รอ → เปลี่ยนร่างที่ค้างเป็น failed → เปิดรับคำขอ ถ้าไม่มี `JWT_SECRET` หรือต่อฐานข้อมูลไม่ได้ เซิร์ฟเวอร์จะไม่ขึ้นเลย

ตามคำขอ 1 ครั้ง: ผู้ใช้ตอบ "ไม่ใช่"

1. หน้าเว็บส่ง `POST /traversal/sessions/:id/actions` พร้อม `{"type":"answer","value":"no"}`
2. `optionalAuth` ตรวจ token ถ้ามี ถ้าไม่มีถือเป็น guest
3. `parseTraversalAction` ตรวจรูปแบบ ถ้าผิดตอบ 400
4. `TraversalService` อ่าน session จาก DB และตรวจสิทธิ์ ถ้าเป็น session ของ user อื่นตอบ 404
5. engine คำนวณโหนดถัดไป (บท 1)
6. บันทึก UPDATE `sessions` และ INSERT `session_history` ใน transaction เดียว
7. ตอบกลับโหนดถัดไป หน้าเว็บเอาไปใส่ cache แล้วแสดงผล

ความปลอดภัยของ SQL

```
const [rows] = await this.pool.query(`${SELECT_USER} WHERE email = ? LIMIT 1`, [email]);
```

ใช้ placeholder `?` ไลบรารีจะ escape ให้ ผู้ใช้จึงแทรกคำสั่ง SQL ไม่ได้ จุดที่ต่อสตริง SQL จะต่อเฉพาะค่าคงที่ในโค้ด ไม่ต่อค่าจากผู้ใช้

กันข้อมูลชนกัน

- `UNIQUE (session_id, step_order)` กัน 2 คำขอซ้อนกันบน session เดียว

- `UPDATE ... WHERE outcome_text IS NULL` ทำให้บันทึกผลลัพธ์ได้ครั้งเดียว
- ใช้ transaction ใน 3 จุด: เดินขั้น, publish ร่าง, seed

รูปแบบ error

```
{ "statusCode": 400, "code": "SAFETY_CONFIRMATION_REQUIRED", "message": "ข้อความไทย" }
```

หน้าเว็บตัดสินจาก `code` ส่วน `message` ใช้แสดงผล และ `asyncRoute` จำเป็นเพราะ Express 4 ไม่ส่ง error ของ async handler ไปที่ errorHandler ให้เอง

endpoint 24 ตัว แบ่งกลุ่ม

กลุ่ม	ตัวอย่าง	สิทธิ์
`/health`	สถานะระบบ	สาธารณะ
`/auth`	register, login, me	สาธารณะ ยกเว้น me
`/symptom-search`	ค้นหา, status	สาธารณะ
`/traversal`	รายการคู่มือ/ผัง, เริ่ม, ตอบ, บันทึกผล, ยกเลิก	guest หรือ user
`/history`	รายการ, รายละเอียด, feedback, ลบ	ต้องรู้เจ้าของ
`/admin`	สถิติ, feedback, ร่างผัง	admin

> **พูดกับกรรมการ: ทำไมย้ายจาก NestJS มา Express**
> "ตอนไล่อ่านโค้ดเตรียมสอบ ส่วนที่ผมอธิบายไม่ได้คือกลไกของ framework เช่น decorator และ dependency injection ไม่ใช่ตรรกะของระบบ Express เป็นฟังก์ชันธรรมดาที่อ่านจากบนลงล่างได้ ตอนย้ายมีเอกสารบันทึกว่าเทียบ HTTP 120 กรณี กับ error 46 ชนิดแล้วตรงกันทั้งหมด แต่ชุดเทียบอยู่นอก repo และบางส่วนยังไม่ได้ทดสอบมือหลังย้าย"

> **ระวัง**
> test อัตโนมัติเหลือไฟล์เดียวคือ `admin-guard.spec.ts` และ error ที่ไม่รู้จักส่ง `message` ดิบกลับไปให้ client ได้ (`error-handler.ts:67-71`)

> **ลองตอบ**
> 1. ทำไมการตอบแต่ละขั้นต้องใช้ transaction
> 2. `?` ใน SQL กัน SQL injection ได้อย่างไร
> 3. Middleware ต่างจาก Service อย่างไร

ดูในโค้ด `backend/src/app.ts:45-85` · `traversal/traversal.routes.ts` · `traversal.service.ts:86-104` · `session.store.ts:89-142` · `common/error-handler.ts:74-115` · `auth/users.store.ts:55-58`

ฐานข้อมูล

มี 3 กลุ่มตาราง: ความรู้, การใช้งาน และงาน admin ซึ่งแยกออกมาเพื่อไม่ให้ผู้ใช้เห็นร่าง

กลุ่ม	ความสัมพันธ์	เก็บอะไร
ความรู้	`manuals` → `graphs` → `nodes` `equipment` → `equipment_usage`	คู่มือ, ผังต่ออาการ, โหนด, อุปกรณ์ที่อาจต้องใช้

การใช้งาน	`users` → `sessions`
	`sessions` → `session_history`
	`sessions` → `session_feedback`
		บัญชี, การตรวจแต่ละครั้ง, ทุกคำตอบ, feedback

งาน admin	`graph_drafts`	ร่างจาก LLM, ข้อความหน้า PDF (ไม่เก็บไฟล์), รายงาน validator

การตัดสินใจสำคัญ

การออกแบบ	เหตุผล
`nodes` ตารางเดียวเก็บทุกชนิด	อ่านง่าย คอลัมน์ที่ชนิดนั้นไม่ใช้เป็น NULL และ PK คือ `(graph_id, node_id)`
ไม่มีตารางประวัติแยก	ประวัติคือ `sessions` ที่ `status='completed'` รวมกับ `session_history`
อายุ session	เดินอยู่ 24 ชั่วโมง ต่ออายุทุกขั้น เมื่อจบเป็น 90 วัน
`session_feedback` ใช้ PK = session_id	1 session มี 1 feedback ส่งซ้ำคือแก้ค่าเดิม (`ON DUPLICATE KEY UPDATE`)
FK `sessions → graphs` แบบ RESTRICT	ลบผังที่มีประวัติใช้งานอยู่ไม่ได้
CHECK ใน `equipment_usage`	ต้องมี `source_page` หรือ `author_note` อย่างน้อยหนึ่งอย่าง คือบังคับว่าต้องมีที่มา
`utf8mb4` ทุกตาราง	เก็บไทยและอิโมจิครบ และ collation ต้องตรงกันจึงสร้าง FK ได้
ไม่เก็บ embedding	คำนวณตอนบูต ไม่มีปัญหาเวกเตอร์ค้าง และ MySQL ใน setup นี้ไม่มี index ค้นเวกเตอร์
UUID เป็น id ของ user/session	เดาไม่ได้เหมือนเลขเรียงต่อกัน

> **ระวัง**
>
> - `nodes` **ไม่มีคอลัมน์เลขหน้ารายโหนด** และไม่มีตาราง embedding
> - migration บน branch นี้คือ 001–006 แต่ `main` ใช้ 12 ไฟล์ชื่อเดิม **อย่ารัน migrate กับ DB บน Aiven** เพราะ runner จำตามชื่อไฟล์
> - DB บน Aiven ใช้ร่วมกับโปรเจกต์เก่า FixStep
> - ไม่มีตัวลบ session หมดอายุ แถวยังอยู่ แค่ทุก query กรอง `expires_at > NOW()` และไม่มี endpoint ลบบัญชี

> **พูดกับกรรมการ: ทำไมใช้ MySQL**
>
> "ข้อมูลเป็นความสัมพันธ์ชัดเจน คู่มือ ผัง โหนด และ session กับคำตอบ ผมคุ้นเคยและใช้บริการฟรีที่มีอยู่แล้ว ไม่ต้องใช้ pgvector เพราะการค้นหาเวกเตอร์ทำในหน่วยความจำ"

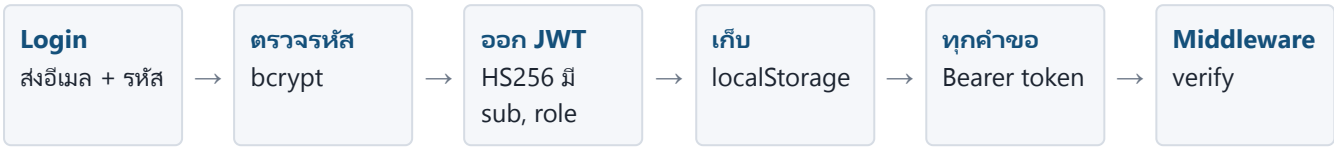
> **ลองตอบ**
>
> 1. ประวัติการซ่อมเก็บที่ตารางไหน
> 2. ทำไมใช้ utf8mb4 แทน utf8
> 3. ถ้าส่ง feedback ซ้ำ session เดิม ข้อมูลเป็นอย่างไร

ดูในโค้ด `backend/migrations/001_users.sql` ถึง `006_graph_drafts.sql` · `002_graphs.sql:53-84` · `004_sessions.sql:17-77` · `003_equipment.sql:35-36` · `scripts/run-migrations.js:33-40`

บท 7

บัญชีผู้ใช้ สิทธิ์ และ guest

ใช้งานหลักได้โดยไม่สมัคร ถ้าล็อกอิน เซิร์ฟเวอร์ให้บัตรผ่าน (JWT) อายุ 7 วัน ส่วนสิทธิ์ admin เช็กจากฐานข้อมูลทุกครั้ง



สิทธิ์ของแต่ละแบบ

ความสามารถ	Guest	User	Admin
ค้นหา เดินผัง บันทึกผล	ได้	ได้	ได้
ประวัติ feedback ลบ	ได้ ผูกกับเบราว์เซอร์	ได้ ผูกกับบัญชี	ได้
หน้า admin	ไม่ได้	ไม่ได้ (403)	ได้

Middleware 3 ตัว

ตัว	ทำอะไร
`optionalAuth`	มี token ก็ตรวจ ผิดตอบ 401 ไม่มีก็ถือเป็น guest
`requireAuth`	ต้องมี token ที่ถูกต้อง
`requireAdmin`	**อ่าน role จาก DB ทุกคำขอ** ไม่เชื่อ role ใน token จึงถอดสิทธิ์แล้วมีผลทันที

`adminArea` ใส่ด่านครั้งเดียวที่ router แม่ของ `/admin` route ใหม่จึงถูกป้องกันอัตโนมัติ และ `AdminRoute` บนหน้าเว็บเป็นแค่ UX ตัวป้องกันจริงอยู่ที่เซิร์ฟเวอร์

รายละเอียดที่พูดแล้วดูเข้าใจจริง

- **dummyHash:** ถ้าไม่พบอีเมล ยัง `compare` กับ hash หลอก เวลาตอบจึงใกล้เคียงกัน และใช้ข้อความ error เดียวกัน กันการเดาว่าอีเมลนี้มีบัญชีผ่านหน้า login
- **ล็อก algorithm:** `jwt.verify` รับเฉพาะ HS256
- **รหัสผ่าน ≤ 72 ไบต์:** bcrypt ใช้แค่ 72 ไบต์แรก และอักษรไทยใช้ 3 ไบต์ต่อตัว
- **สมัครได้แค่ user:** ส่ง `role` มาใน body ก็ไม่มีผล admin สร้างได้ทาง `npm run seed:admin` เท่านั้น
- **Guest:** เบราว์เซอร์สุ่ม UUID เก็บใน localStorage ส่งเป็น `X-Client-Id` SQL แยกเจ้าของด้วย `client_id = ? AND user_id IS NULL`

จุดอ่อนที่รู้อยู่แล้ว

จุดอ่อน	ถ้าแก้
login ไม่มี rate limit	`express-rate-limit`
token อยู่ใน localStorage ถ้ามี XSS ถูกขโมยได้	httpOnly cookie + CSRF token

logout ไม่เพิกถอน token ใช้ได้ถึงหมดอายุ	ลดอายุ + refresh token
session ของ guest ที่กำลังเดินไม่ตรวจเจ้าของ แต่ต้องรู้ UUID ของ session	ตรวจ `client_id` ทุกคำขอ
ไม่มี helmet และ error 500 ส่งข้อความดิบ	เพิ่ม helmet และส่งข้อความกลาง
ประวัติ guest หายเมื่อเปลี่ยนเบราว์เซอร์และไม่ย้ายเข้าบัญชีตอนล็อกอิน	เพิ่ม flow ผูกประวัติ

พูดกับกรรมการ

"จุดที่ผมตั้งใจคือสิทธิ์ admin อ่านจากฐานข้อมูลทุกครั้ง ไม่เชื่อ token และใช้ dummy hash กันการเดาอีเมลตอน login จุดอ่อนที่รู้อยู่คือยังไม่มี rate limit และ token อยู่ใน localStorage ผมมีแผนแก้ไว้แล้ว"

ลองตอบ

1. ถอดสิทธิ์ admin ขณะที่ token ยังไม่หมดอายุ เข้า `/admin` ได้ไหม เพราะอะไร
2. Guest ระบุตัวอย่างไร ถือเป็นการยืนยันตัวตนไหม
3. Logout แล้ว token เดิมยังใช้ได้ไหม

ดูในโค้ด `backend/src/auth/token.ts` · `auth.middleware.ts:54-87` · `auth.service.ts:26, 55-63` · `session-owner.ts:25-30` · `admin/admin.routes.ts:9-14` · `frontend/src/lib/clientId.ts`

บท 8

Frontend

เป็น SPA ไม่มี Redux ข้อมูลจากเซิร์ฟเวอร์ให้ TanStack Query จัดการ ข้อมูลผู้ใช้อยู่ใน Context สถานะเล็กๆ ของหน้าใช้ `useState`

เส้นทางหลัก

Path	หน้าที่
`/devices`	เลือกเครื่องและรุ่น หรือ "ไม่ทราบรุ่น"
`/symptoms` , `/symptoms/all`	ค้นหา และดูอาการทั้งหมด ถ้ายังไม่เลือกรุ่นจะพากลับ `/devices`
`/session/:sessionId`	เดินผังทีละขั้น
`/history` , `/history/:sessionId`	ประวัติ feedback และลบ
`/login` , `/register`	บัญชี
`/admin/analytics` , `/feedback` , `/drafts`	สถิติ feedback และร่างผัง

State อยู่ที่ไหน

ชนิด	เก็บที่
ข้อมูลจากเซิร์ฟเวอร์	TanStack Query cache
ผู้ใช้ที่ล็อกอิน	React Context + localStorage `fixstep.auth`
Guest id	localStorage `fixstep.clientId`
รุ่นที่เลือก	URL query string จึงรีเฟรชแล้วไม่หาย
กันกดซ้ำ	`useRef` (`isSendingRef`) เพราะอัปเดตทันที ไม่รอ render

TanStack Query ช่วยอะไร

- **Query key มีตัวตนผู้ใช้:** เช่น `['history', identity, 'list', page]` เปลี่ยนผู้ใช้แล้วไม่เห็นข้อมูลคนก่อน และล้าง cache ทุกครั้งที่ login/logout
- **ลดการโหลดซ้ำ:** หลังตอบแต่ละขั้น ใช้ `setQueryData` ใส่ผลลง cache ทันที ไม่ต้อง GET ใหม่
- **Polling เฉพาะที่จำเป็น:** poll ร่างทุก 3 วินาที เฉพาะตอนมีร่าง `drafting`
- **ค้นหาเป็น mutation:** ผลค้นหาไม่ถูก cache

Component สำคัญ

Component	หน้าที่
`StepView`	เลือกการแสดงผลตามชนิดโหนด
`SafetyGateStep`	กล่องเตือนแดงและ checkbox บังคับ

`ReferenceLine`	บรรทัดที่มาของผัง
`RephraseNote`	บอกว่าข้อความถูกปรับด้วย AI และดูต้นฉบับ
`SymptomSearchResults`	แสดงผลไม่เกิน 3 พร้อมคะแนน หรือข้อความ "ไม่เดา"
`DraftJsonEditor`, `ValidationReportPanel`	ตรวจและแก้ร่าง

การเรียก API

ทุกคำขอผ่าน `request()` ตัวเดียวใน `http.ts` ใส่ token ให้อัตโนมัติ แปลง error เป็น `ApiError` และเมื่อได้ 401 ขณะมี token จะออกจากระบบให้ `describeError` แปลง `code` เป็นข้อความไทย ส่วนฝั่ง client ตรวจความยาวก่อนส่งเพื่อความสะดวก แต่เซิร์ฟเวอร์ตรวจซ้ำเสมอ

> **ระวัง**
> ชื่อบนหน้าจอยังเป็น "FixStep" จากโปรเจกต์ก่อน ถ้ากรรมการถาม ให้ตอบตรงๆ ว่ายังไม่ได้เปลี่ยน ไม่มีผลต่อการทำงาน และไม่มีไฟล์เทสต์ frontend

> **พูดกับกรรมการ: ทำไมใช้ TanStack Query**
> "ข้อมูลส่วนใหญ่อยู่ที่เซิร์ฟเวอร์ ไม่ใช่สถานะของหน้าจอ TanStack Query จัดการโหลด error และ cache ให้ จึงไม่ต้องเขียนเอง และไม่ต้องใช้ Redux ผมใส่ตัวตนผู้ใช้ไว้ใน query key เพื่อไม่ให้ข้อมูลคนก่อนปนมา"

> **ลองตอบ**
> 1. ทำไมเก็บรุ่นที่เลือกไว้ใน URL
> 2. ทำไมกันกดซ้ำด้วย `useRef` แทน `useState`
> 3. การล็อกปุ่มบนหน้าเว็บอย่างเดียวพอไหมสำหรับขั้นอันตราย

ดูในโค้ด `frontend/src/App.tsx:37-51` · `api/queries.ts` · `api/http.ts:39-86` · `components/step/StepView.tsx` · `pages/SessionPage.tsx:42-57`

บท 9

ประวัติ feedback และ dashboard

ทุกครั้งที่ตรวจจนจบ รายการจะอยู่ในประวัติ 90 วัน ผู้ใช้ให้ feedback ได้ และ admin ดูภาพรวมจาก dashboard

ประวัติการซ่อม

- **รายการ:** `sessions` ที่จบแล้วและยังไม่หมดอายุของเจ้าของนั้น ดึงล่าสุดไม่เกิน 50 รายการ แบ่งหน้าละ 10
- **รายละเอียด:** ถ้าไม่ใช่ของตัวเอง ตอบ 404 โดยไม่บอกว่ามีอยู่
- **เล่นซ้ำ (replay):** นำคำตอบที่บันทึกไว้เดินผ่าน engine บนผังปัจจุบัน ถ้าไม่จบที่โหนดเดิม แปลว่าผังถูกแก้ ระบบจะแสดง `available: false` แทนการแสดงขั้นเก่าคู่กับข้อสรุปที่ไม่เกี่ยวกัน
- **ลบ:** คำสั่ง DELETE เดียวที่มีเงื่อนไขเจ้าของ `session_history` และ `session_feedback` หายตาม CASCADE

> **จุดเด่นที่พูดได้**
> "Replay ทำได้เพราะ engine เป็น pure function ถ้าผังเปลี่ยนจนเส้นทางไม่ตรง ระบบจะไม่แสดงข้อมูลที่ขัดกัน"

Feedback

```
INSERT INTO session_feedback (session_id, helpful, `comment`) VALUES (?, ?, ?)
ON DUPLICATE KEY UPDATE helpful = VALUES(helpful), `comment` = VALUES(`comment`)
```

1 session มี 1 feedback ส่งซ้ำคือแก้ค่าเดิม (upsert) comment ไม่เกิน 500 ตัวอักษร และให้ได้เฉพาะ session ที่จบแล้ว ส่วน `outcome_text` เป็นอีกอย่าง คือข้อความที่ผู้ใช้พิมพ์ตอนจบ บันทึกได้ครั้งเดียว

Dashboard: `GET /admin/analytics/summary?days=1..90`

ตัวเลข	ความหมาย
sessions	ทั้งหมด จบแล้ว กำลังเดิน ทิ้งกลางทาง และแยกตาม user / guest
confidence	ค่าเฉลี่ย cosine ของ session ที่มีคะแนน (NULL ไม่ถูกนับ)
feedback	satisfactionRate = helpful / total ถ้าไม่มี feedback เป็น null ไม่ใช่ 0
outcomes	แยกผลจบตาม `outcome_kind` ของโหนดสุดท้าย
topGraphs, topDevices	อาการและประเภทเครื่องที่ถูกตรวจบ่อย 10 อันดับ
perDay	เริ่มและจบต่อวันตามเวลาไทย เติม 0 ในวันที่ไม่มีข้อมูล

เซิร์ฟเวอร์ยิง 5 query พร้อมกันด้วย `Promise.all` และหยิบเฉพาะช่องที่ DTO ระบุ ไม่ส่งข้อมูลอย่างอีเมลออกไป

> **ระวัง**
> - satisfaction rate มาจากคนที่เลือกกด feedback (selection bias) **ไม่ใช่ความแม่นยำของระบบ**
> - ตัวเลขตอนเดโมเป็นข้อมูลทดสอบ ไม่ใช่ผู้ใช้จริง ต้องพูดเองก่อนถูกถาม
> - ประวัติเกิน 50 รายการจะเห็นแค่ 50 ล่าสุด และ `total` ไม่เกิน 50

ลองตอบ

1. Replay คืออะไร ทำไมต้องมี
2. ไม่มี feedback เลย ทำไม satisfactionRate เป็น null ไม่ใช่ 0
3. satisfaction 90% แปลว่าระบบแม่น 90% ไหม

ดูในโค้ด `backend/src/history/history.store.ts:48-60, 95-112` · `history.service.ts:98-156, 196-221` · `admin/admin.service.ts:52-124` · `admin.store.ts:111-192`

การประเมินผล

ณ วันที่จัดทำ ระบบยังไม่มีผลประเมินความแม่นยำที่อ้างได้ และยังไม่ได้ทดสอบกับผู้ใช้จริง บทนี้ช่วยให้ตอบตรงๆ โดยไม่ดูอ่อน

สิ่งที่มี และที่อ้างไม่ได้

สิ่งที่มี	ผล	ข้อจำกัด
ตรวจผังกับ PDF	FAIL 0 ตรวจไม่ได้ 48 ผัง (ตัดแอร์แล้ว)	วัดคำและตัวเลข ไม่ใช่ความหมาย
ตัวอย่างคะแนนค้นหา	6 คำค้น	เป็นตัวอย่าง ไม่ใช่ชุดทดสอบ
ชุดตั้งเกณฑ์ 56 ข้อ	top-1 29/39, top-3 37/39, นอกขอบเขตหลุด 8/17	**ห้ามอ้าง** AI ร่าง และเป็นคู่มือแอร์ที่ไม่อยู่ในระบบแล้ว
Feedback ในระบบ	มีเครื่องมือเก็บ	ยังไม่มีผู้ใช้จริง

ตรวจเมื่อ 31 ต.ค.: ยังไม่มีโฟลเดอร์ `data/eval/` ใน repo

สิ่งที่ควรวัด

ระบบไม่ได้วินิจฉัย แต่หาผังที่ใช่ จึงควรวัดคุณภาพการเลือกผัง และการปฏิเสธเมื่อไม่มีผังที่ใช่

Metric	นิยาม
Top-1 hit rate	ในขอบเขต: อันดับ 1 เป็นผังที่ถูกและถึงเกณฑ์
Top-3 hit rate	ในขอบเขต: ผังที่ถูกอยู่ในผลที่แสดง ตรงกับการใช้งานจริงที่ผู้ใช้เลือกเอง
False rejection	ในขอบเขต แต่ระบบตอบว่าไม่พบ
Out-of-scope rejection	นอกขอบเขต และระบบตอบว่าไม่พบ นี่คือความสามารถ "ไม่เดา"

ถ้ามีเวลาทำชุดเล็ก

1. เลือก 4 เล่ม (GE83MDT, RB6000D, KM2DVBADN, WA5000R) เล่มละ 10–15 คำค้นในขอบเขต และ 5 คำค้นนอกขอบเขต ห้ามลอกคำพ้องตรงตัว
2. ตั้งเฉลย **ก่อน**ดูผลของระบบ แล้วแบ่งชุด tuning กับชุด report
3. กวาด threshold 0.40–0.90 บน tuning เลือกค่าที่ TPR + TNR − 1 สูงสุด (Youden's J)
4. รายงานผล 0.55 กับค่าใหม่บนชุด report พร้อมข้อจำกัด: ชุดเล็ก ผู้พัฒนาตั้งเฉลยเอง 4 จาก 21 เล่ม

การทำชุดนี้ไม่ต้องแก้ระบบ ใช้โค้ดที่ build แล้วและไฟล์ JSON ได้ ร่างสคริปต์อยู่ในไฟล์ `10-evaluation.md` ถ้าไม่ทัน ให้ใช้คำตอบด้านล่าง และเสนอเป็นงานถัดไป

> **พูดกับกรรมการ: ถ้ายังไม่มีผล**
> "ยังไม่ได้ทดสอบกับผู้ใช้จริง และยังไม่มีชุดทดสอบการค้นหาที่อ้างเป็นความแม่นยำได้ครับ ผมไม่อยากอ้างตัวเลขที่ไม่มี สิ่งที่ทำแล้วคือตรวจข้อมูลในผังกับ PDF ด้วยสคริปต์ และแผนต่อไปคือวัด top-3 hit rate กับ out-of-scope rejection ด้วยชุดที่ตั้งเฉลยล่วงหน้า แล้วใช้ผลมาปรับ threshold จากนั้นทำ usability test 5–8 คน วัด task success และ SUS"

ถ้าถูกถามต่อ: "แล้วรู้ได้ไงว่าใช้ได้จริง"

"ระบบไม่ได้สร้างคำแนะนำเอง ความถูกต้องของคำแนะนำจึงขึ้นกับผังเทียบกับคู่มือ ซึ่งตรวจได้โดยไม่ต้องมีผู้ใช้ ความเสี่ยงที่เหลือคือเลือกผังผิด ซึ่งผมลดด้วยการให้ผู้ใช้เลือกเอง แต่ยอมรับว่ายังไม่ได้วัดว่าผู้ใช้ทั่วไปตอบคำถามในผังถูกไหม เช่น 'ฟิวส์ขาดไหม' อาจตอบไม่ได้"

ระวัง

ห้ามอ้าง 37/39 เป็นความแม่นยำของระบบ และห้ามใช้ satisfaction rate แทนความแม่นยำ

ลองตอบ

1. ทำไม top-3 ตรงกับการใช้งานจริงมากกว่า top-1
2. ทำไมต้องแยกชุด tuning กับ report
3. ทำไมต้องวัด out-of-scope rejection ด้วย

ดูเพิ่ม `docs/exam-prep/10-evaluation.md` · `backend/scripts/study-measure.js` · `docs/STUDY_GUIDE.md:604-629`

บท 11

Deployment และเดโมจากเครื่อง

ระบบออกแบบให้ deploy บนบริการฟรี 3 ส่วน แต่สิ่งที่ออนไลน์อยู่เป็นเวอร์ชันเก่า จึงเลือกเดโมจากเครื่อง

ส่วน	ตั้งใจไว้	สถานะที่ตรวจล่าสุด
Backend	Render (free)	branch `main` ยังเป็น NestJS (`f9b7e09`)
Frontend	GitHub Pages	build วันที่ 14 ต.ค. ไม่มีหน้าเลือกรุ่น ประวัติ login admin
ฐานข้อมูล	Aiven MySQL	ใช้ร่วมกับโปรเจกต์เก่า FixStep
เดโมจริง	เครื่องตัวเอง	MySQL ในเครื่อง + Express + Vite จากโค้ดที่เปิดให้ดู

รู้ไว้เรื่องการ deploy

- ไม่มี Docker ใช้ native Node runtime ผ่าน `render.yaml` เพื่อลดงาน ops
- Render free หลับเมื่อไม่มีคนใช้ และปลุกครั้งแรกใช้เวลาราว 1 นาที
- ไฟล์โมเดลอยู่ใน git จึงไม่ต้องดาวน์โหลดตอนรัน
- ค่าลับอยู่ใน env ไฟล์ `.env` ไม่อยู่ใน git และ `VITE_*` จะถูกฝังใน JS จึงห้ามใส่ความลับ

เตรียมเดโมบนเครื่อง

ขั้นตอนเต็มอยู่ในไฟล์ `17-local-demo-runbook.md` สรุปลำดับคือ:

1. DB ใหม่ ชื่อ appliance_demo → **2. .env** สำรองของเดิมก่อน → **3. migrate + seed** ได้ 631 ผัง → **4. backend** รอ "ระบบค้นหาพร้อม" → **5. frontend** localhost:5173

> **ระวัง: ต้องซ้อมก่อนวันสอบ**
>
> คำสั่งใน runbook **ยังไม่เคยรันจริง** ค่าที่ "ควรเห็น" มาจากการอ่านโค้ด ถ้าเจอปัญหาจะได้แก้ทันก่อนวันสอบ และห้ามชี้ไป DB บน Aiven

สคริปต์เดโมแบบย่อ (ประมาณ 6 นาที)

ช่วง	ทำบนจอ	ประเด็นที่พูด
เคสปกติ	ไมโครเวฟ → GE83MDT → "ไมโครเวฟไม่ร้อน" → เลือก "อาหารไม่สุกเลย" → `n1` ไม่ใช่ → `n1f` ติ๊กเตือน → `n1v` ใช่ → จบ	คะแนนไม่ใช่โอกาสถูก, บรรทัดอ้างอิงคือหน้าของหัวข้อ, ขั้นอันตรายบังคับที่เซิร์ฟเวอร์
ข้อมูลไม่พอ	พิมพ์ "วันนี้ฝนตก"	ไม่ถึงเกณฑ์ ระบบไม่เดา
ประวัติ	เปิดรายการ → กด 👍 + comment → เปลี่ยนเป็น 👎	1 session มี 1 feedback, replay ผ่าน engine
Admin	`/admin/analytics` → `/admin/feedback` → `/admin/drafts` ร่างที่เตรียมไว้	ตัวเลขเป็นข้อมูลทดสอบ, LLM เป็นแค่ผู้ร่าง, คนต้องกด publish
ปิด	สไลด์ข้อจำกัด	พูดข้อจำกัดเองก่อนถูกถาม (บท 12)

แผนสำรองเมื่อไม่มีเน็ต

ค้นหา เดินผัง ประวัติ feedback และ dashboard ทำงานในเครื่องได้ทั้งหมด สิ่งที่ต้องใช้เน็ตมีแค่ rephrase และการร่างผังใหม่

1. ตั้ง `REPHRASE_ENABLED=false` แล้วรีสตาร์ต backend ถ้าเปิดไว้ ขั้นที่ไม่ใช่ขั้นอันตรายอาจรอสูงสุด 3 วินาทีก่อนใช้ข้อความต้นฉบับ
2. ใช้ร่าง `ready` ที่เตรียมไว้แทนการอัปโหลดสด
3. พูดว่า "นี่คือเหตุผลที่ไม่ให้ LLM อยู่บนเส้นทางหลัก"

ก่อนกรรมการเห็นหน้าจอ

- ปิดแท็บ `.env` ห้ามพิมพ์ค่าในเทอร์มินัล และปิดแท็บ Aiven, Render, Groq, อีเมล, LINE
- อย่าเปิด DevTools แท็บ Application หรือ Network เพราะมี JWT
- ใช้บัญชีเดโมเท่านั้น และเปิด Focus Assist
- ดู `git log -1 --oneline` ให้รู้ว่าเดโมคือ commit ไหน

> **พูดกับกรรมการ: ระบบที่เดโมตรงกับโค้ดไหม**
>
> "ตรงครับ เดโมนี้รันบนเครื่องทั้งระบบจากโค้ดที่เปิดให้ดู ส่วน URL สาธารณะยังเป็นเวอร์ชันก่อนหน้า ผมจึงเลือกเดโมจากเครื่องเพื่อให้สิ่งที่เห็นตรงกับที่อธิบาย และไม่เสี่ยงกับ DB ที่ใช้ร่วมกับโปรเจกต์อื่นก่อนสอบ"

> **ลองตอบ**
>
> 1. ทำไมไม่เดโมจาก URL ที่ deploy ไว้
> 2. ถ้าเน็ตล่มกลางเดโม อะไรยังทำงาน
> 3. ทำไมห้ามรัน migrate กับ DB บน Aiven

ดูเพิ่ม `docs/exam-prep/17-local-demo-runbook.md` · `15-demo-script.md` · `11-deployment.md` · `render.yaml`

บท 12

ข้อจำกัด และงานในอนาคต

หลักการตอบ: บอกข้อจำกัดก่อนถูกถาม แล้วตามด้วย "ระบบรับมืออย่างไร" และ "จะแก้อย่างไร" การรู้จุดอ่อนของตัวเองเป็นหลักฐานว่าเข้าใจระบบจริง

ข้อจำกัดและคำตอบ

ข้อจำกัด	ตอบอย่างไร	ถ้าแก้
ครอบคลุม 21 คู่มือ	ไม่มีผังก็บอกว่าไม่พบและแนะนำศูนย์บริการ ไม่เดา	ใช้ทาง B เพิ่มคู่มือ
Citation อยู่ระดับผัง	บรรทัดอ้างอิงเป็นหน้าของหัวข้อ เลขหน้ารายคำสั่งอยู่ในข้อความ (instruction 89%)	เพิ่มเลขหน้ารายโหนด
ผู้ใช้ไม่เห็นว่าข้อความไหนผู้พัฒนาเพิ่ม	มีบันทึกที่มาในข้อมูลแล้ว แต่ยังไม่แสดงบนหน้าจอ	แสดงป้ายจาก `author_note`
คำแปลและลำดับชั้นยังไม่มีคนตรวจ	ตรวจสิ่งที่โปรแกรมตรวจได้แล้ว ส่วนความหมายต้องใช้คน	สุ่มตรวจ + เก็บสถานะผู้ตรวจ
ยังไม่มีผลประเมิน	ดูบท 10	ชุดทดสอบ + usability test
Threshold ไม่ได้ปรับ	มาจากชุดเก่า ภายในรุ่นแยกได้ไม่ดี ระบบจึงให้ผู้ใช้เลือกเอง	กวาดค่าใหม่, threshold ต่อรุ่น, hybrid search
ค้นหาแบบ brute force	พอสำหรับข้อมูลขนาดนี้ เวกเตอร์ใช้ RAM ราว 3 MB	HNSW หรือ vector DB
ยืนยันความปลอดภัยคือการติ๊ก	ระบบรู้ไม่ได้ว่าทำจริง แต่บังคับให้เห็นคำเตือนก่อน	ไม่มีทางแก้ด้วยซอฟต์แวร์ล้วน
ไม่ใช่การวินิจฉัยยืนยัน	พาตรวจตามคู่มือเพื่อหาสาเหตุที่เป็นไปได้ และผลขึ้นกับคำตอบของผู้ใช้	เพิ่มข้อความปฏิเสธความรับผิดชอบ
เทสต์เหลือไฟล์เดียว	ตอนย้าย framework ใช้ชุดเทียบนอก repo ยอมรับว่าควรมีเพิ่ม	นำเทสต์ engine/matcher/guard กลับมา
Security ยังไม่ครบ	บท 7 มีรายการและแผนแก้	rate limit, helmet, cookie

Citation รายโหนด: ตัวเลขที่ควรรู้

ชนิดโหนด	ทั้งหมด	มี "คู่มือหน้า N" ในข้อความ
instruction	1,107	980 (89%)
escalation	780	602 (77%)
resolution	706	193 (27%)
checkpoint	2,146	10 (0.5%)

มี 541 โหนดอ้างหลายหน้า และ 860 โหนดอ้างหน้าที่อยู่นอกช่วงหน้าของผัง เช่น ผังที่หัวข้ออยู่หน้า 37 แต่ขั้นตอนอ้างหน้า 3, 10 หรือ 40 ป้ายบนจอที่เขียนว่า "ที่มาของขั้นตอนนี้" จึงยังไม่ตรงความหมาย

> **พูดกับกรรมการ: citation**
>
> "บรรทัดอ้างอิงเป็นหน้าของหัวข้อแก้ปัญหา ส่วนเลขหน้าของแต่ละคำสั่งอยู่ในวงเล็บในข้อความ ประมาณ 89% ของขั้นที่ให้ลงมือทำ ยังไม่ใช่ citation รายโหนดแบบมีโครงสร้าง และป้ายบนจอยังสื่อเกินจริงเล็กน้อย ผมรู้ปัญหานี้ และวางแผนเพิ่มเลขหน้าเป็นข้อมูลของแต่ละโหนด"

งานในอนาคต (เรียงตามความคุ้มค่า)

1. ประเมินผลจริง: ชุดทดสอบ และ usability test
2. Citation รายโหนด และแสดงป้ายที่มาจาก `author_note`
3. ให้คนตรวจคำแปลและลำดับขั้น
4. ขยายคู่มือ ยี่ห้อ และประเภทเครื่อง โดยเปลี่ยน ENUM เป็นตารางอ้างอิง
5. ปรับการค้นหา: hybrid search, re-ranker, threshold ต่อรุ่น
6. Security และ ops: rate limit, helmet, Docker, CI, แยก DB จากโปรเจกต์เก่า
7. ใช้ feedback 👎 ชี้ผังที่ควรแก้ และรองรับรูปภาพในขั้นตอน

> **ลองตอบ**
>
> 1. ทำไมควรพูดข้อจำกัดเองก่อนถูกถาม
> 2. บรรทัด "หน้า 37" ตรงกับทุกขั้นในผังไหม
> 3. ถ้ามีเวลาอีก 1 เดือน จะทำอะไรก่อน เพราะอะไร

ดูเพิ่ม `docs/exam-prep/12-limitations-and-future-work.md` · `14-weak-points-and-risks.md` · `18-proposal-author-note-citation.md`

ตัวเลขที่ต้องจำ

เรื่อง	ค่า
คู่มือ / ผัง / โหนด	21 / 631 / 4,739
ประเภทเครื่อง	ไมโครเวฟ 6, ตู้เย็น 5, ทีวี 5, เครื่องซักผ้า 5
โหนด safety_critical	368
โหนดที่มี author_note	1,264 (27%) เป็นคำเชื่อม 720
ป้ายในดัชนี / ข้อความไม่ซ้ำ	3,814 / 1,936
Embedding	MiniLM หลายภาษา, q8, 384 มิติ, ไฟล์ ~56 MB
Threshold / ผลสูงสุด	0.55 / 3
RAM	เวกเตอร์ ~3 MB, ทั้ง process ~250–300 MB (ประมาณการ)
Rephrase	ปิดเป็นค่าเริ่มต้น, temperature 0, timeout 3 วินาที
ร่างผัง	2 หน้าต่อครั้ง, เลือก ≤ 10 หน้า, PDF ≤ 10 MB, 24 กฎ error + 5 กฎ warning
JWT / bcrypt	HS256 อายุ 7 วัน / cost 10
อายุ session	เดินอยู่ 24 ชั่วโมง, จบแล้ว 90 วัน
Endpoint / migration	24 / 001–006
ผลตรวจกับ PDF	FAIL 0, ตรวจไม่ได้ 48 ผัง

สิ่งที่ห้ามพูด

อย่าพูดว่า	ให้พูดว่า
"ระบบนี้เป็น RAG"	"Retrieval-guided state machine LLM ไม่ได้เขียนคำแนะนำ"
"คะแนน 0.82 แปลว่าถูก 82%"	"เป็นความคล้ายของข้อความ ไม่ใช่ความน่าจะเป็น"
"ระบบไม่มีทาง hallucinate"	"ไม่แต่งสดบนเส้นทางหลัก แต่ข้อมูลตอนเตรียมยังมีความเสี่ยง"
"ทุกขั้นบอกหน้าของตัวเอง"	"บรรทัดอ้างอิงเป็นหน้าของหัวข้อ เลขหน้ารายคำสั่งอยู่ในข้อความ"
"ทุกข้อความมาจากคู่มือ"	"ส่วนใหญ่มาจากคู่มือ และมีส่วนที่ผมเพิ่มพร้อมบันทึกที่มา"
"81 โหนดไม่ได้มาจากคู่มือเลย"	"คำเตือนทั้งก้อน 36 โหนด และบางส่วนอีกราว 45 โหนด"
"คำเตือนที่เพิ่มเข้มกว่าคู่มือทุกข้อ"	"ยังไม่ได้เทียบทีละข้อ และยังไม่มีผู้เชี่ยวชาญตรวจ"
"ความแม่นยำ 37/39" หรือ "พึงพอใจ 90%"	"ยังไม่มีผลประเมินที่อ้างได้"
"คนตรวจครบ 631 ผังแล้ว"	"ตรวจอัตโนมัติแล้ว แต่คำแปลและลำดับยังไม่มีคนตรวจเป็นระบบ"
"ระบบที่ deploy คือเวอร์ชันนี้"	"URL สาธารณะยังเป็นเวอร์ชันก่อน ผมเดโมจากเครื่อง"
"ระบบวินิจฉัยเครื่องเสียได้"	"พาตรวจตามคู่มือเพื่อหาสาเหตุที่เป็นไปได้"

คำถามยากที่ควรซ้อม

คำถาม	แนวตอบ
0.55 มาจากไหน	มาจากชุดเก่าของคู่มือแอร์ที่ AI ร่าง ยังไม่ได้ปรับกับข้อมูลปัจจุบัน จึงไม่อ้างว่าเป็นค่าที่ดีที่สุด
พิมพ์ "ไมโครเวฟเสีย" ทุกผังคงผ่านเกณฑ์ ระบบช่วยอะไร	ใช่ ภายในรุ่นคะแนนสูงกันหมด ระบบให้ 3 อันดับแรก และมีหน้าอาการทั้งหมด
ทำไมไม่ใช้ ChatGPT	ตรวจไม่ได้ว่าจะตอบอะไร อาจแต่งขั้นตอนหรือละคำเตือน และมีค่าใช้จ่ายกับการส่งข้อมูลออกนอกระบบ
ทำไมไม่ทำ RAG	RAG ลดการแต่งได้แต่ไม่หมด และคำเตือนบังคับด้วย prompt ได้แค่ขอร้อง งานนี้มีความเสี่ยงด้านความปลอดภัย
ทำไมไม่ fine-tune	ข้อมูลไม่พอ ไม่รับประกันว่าไม่แต่ง และต้องเทรนใหม่ทุกครั้งที่เพิ่มคู่มือ
ทำไมไม่ใช้ LangChain	ใช้ LLM แค่ 2 จุด จุดละ HTTP call เดียว เขียนเองอ่านและคุมได้ทุกบรรทัด
ทำไมไม่ใช้ keyword search	ผู้ใช้พิมพ์ภาษาพูด ผังมีสองภาษา embedding เทียบความหมายข้ามภาษาได้ แต่ยังไม่ได้วัดเทียบกัน
ทำไมไม่ใช้ ORM	อยากเห็นและอธิบาย SQL ทุกคำสั่ง ข้อแลกคือต้องแปลงชนิดข้อมูลเอง
ถ้าผู้ใช้ทำตามแล้วเกิดอันตราย ใครรับผิดชอบ	ระบบแนะนำตามคู่มือและมีคำเตือนบังคับ แต่ยังไม่มีข้อความปฏิเสธความรับผิดชอบบนหน้าจอ ถ้าใช้จริงต้องเพิ่ม
ผู้ใช้ตอบผิด ระบบรู้ไหม	ไม่รู้ ผลขึ้นกับคำตอบ เช่น "ฟิวส์ขาดไหม" อาจตอบไม่ได้ ยังไม่ได้วัด จึงเป็นเหตุผลของ usability test
ย้าย framework ใกล้สอบไม่เสี่ยงเหรอ	เสี่ยงครับ จึงเทียบกับชุดเดิม แต่ยอมรับว่าการอัปโหลด PDF และ Groq จริงยังไม่ได้ทดสอบมือหลังย้าย
ส่วนไหนที่คุณภูมิใจ และส่วนไหนอยากทำใหม่	ภูมิใจ: แยกการหาออกจากการตอบ และบังคับความปลอดภัยด้วยโค้ด · อยากทำใหม่: เก็บเลขหน้ารายโหนดตั้งแต่ต้น และประเมินผลให้เร็วกว่านี้ (ปรับเป็นความเห็นของตัวเองได้)

รวม 106 คำถามพร้อมแนวตอบอยู่ในไฟล์ `13-committee-qa.md`

เฉลย "ลองตอบ"

บท 1

1. เหมือนกัน เพราะ engine เป็น deterministic ดูแค่โหนดปัจจุบันกับคำตอบ ไม่มีการสุ่มหรือ AI
2. ได้ 400 `SAFETY_CONFIRMATION_REQUIRED` ล็อกปุ่มไม่พอเพราะยิง API ตรงข้ามหน้าเว็บได้
3. การเลือกโหนดถัดไปใช้ค่าในผังเท่านั้น LLM ไม่ได้อยู่บนเส้นทางนี้

บท 2

1. q·d = 1, ‖q‖ = 1, ‖d‖ = √2 ได้ 1/√2 ≈ 0.707 ผ่าน
2. ไม่ใช่ เป็นความคล้ายระหว่างข้อความ ไม่ได้ calibrate และ engine ไม่ได้ใช้ตัดสิน
3. ป้ายซ้ำกันข้ามผังหรือข้ามรุ่น ข้อความที่เหมือนกันจึงใช้เวกเตอร์ร่วมกัน

บท 3

1. ไม่เป็น RAG ใช้ LLM เขียนคำตอบจากข้อความที่ค้นได้ ระบบนี้ค้นเพื่อเลือกผัง แล้วคำแนะนำมาจากผัง
2. ด่านชุดตัวเลข เพราะ 30 กับ 60 ไม่ตรงกัน ระบบจึงทิ้งผลแล้วใช้ต้นฉบับ
3. ค้นหา เดินผัง ประวัติ feedback dashboard ใช้ได้ ส่วน rephrase กลับเป็นข้อความต้นฉบับ และร่างผังใหม่ใช้ไม่ได้

บท 4

1. ไม่ใช่ ทั้งหมดมาจาก pipeline นอกระบบ (ทาง A)
2. จะผ่าอาการกลางคัน หน่วยที่ต้องการคือหนึ่งอาการต่อหนึ่งผัง
3. validator รายงาน `E_UNREACHABLE` เมื่อมี error ระบบไม่ให้ publish ตอบ 409 `DRAFT_INVALID` จนกว่าจะแก้

บท 5

1. ต้องเปลี่ยนตำแหน่งปัจจุบันและเพิ่มประวัติพร้อมกัน ถ้าสำเร็จแค่อย่างเดียวข้อมูลจะไม่ตรงกัน
2. ค่าถูกส่งแยกจากตัวคำสั่งและถูก escape จึงเป็นข้อมูล ไม่ถูกตีความเป็นคำสั่ง SQL
3. Middleware ทำงานร่วมกับหลาย route ก่อนถึงงานหลัก เช่น ตรวจสิทธิ์ ส่วน Service เก็บตรรกะของงานนั้น

บท 6

1. ไม่มีตารางแยก คือ `sessions` ที่จบแล้วรวมกับ `session_history`
2. `utf8` เดิมของ MySQL เก็บได้ 3 ไบต์ ส่วน `utf8mb4` เก็บอักขระ 4 ไบต์ เช่น อีโมจิ ได้ด้วย
3. มีแถวเดียว ค่าใหม่แทนค่าเดิม เพราะ PK คือ session_id

บท 7

1. ไม่ได้ `requireAdmin` อ่าน role จาก DB ทุกคำขอ จึงได้ 403
2. เบราว์เซอร์สุ่ม UUID ส่งผ่าน `X-Client-Id` ไม่ใช่การยืนยันตัวตน ใครรู้ UUID ก็เข้าถึงประวัติของ guest นั้นได้ แต่เดายากมาก
3. ใช้ได้จนหมดอายุ เพราะ logout แค่ลบฝั่ง client ไม่มีการเพิกถอน

บท 8

1. รีเฟรชหรือแชร์ลิงก์แล้วยังรู้ว่าเลือกรุ่นไหน
2. `useRef` เปลี่ยนค่าทันทีโดยไม่รอ render จึงกันการกดเร็วๆ 2 ครั้งได้

3. ไม่พอ ต้องบังคับที่เซิร์ฟเวอร์ด้วย ซึ่ง engine ทำอยู่แล้ว

บท 9

1. นำคำตอบที่บันทึกไว้เดินซ้ำบนผังปัจจุบัน เพื่อยืนยันว่ายังจบที่โหนดเดิม ถ้าไม่ใช่จะไม่แสดงข้อมูลที่ขัดกัน
2. เพราะ "ยังไม่มีข้อมูล" ต่างจาก "ไม่มีใครพอใจเลย"
3. ไม่ มาจากคนที่เลือกกด feedback และไม่ได้วัดว่าเลือกผังถูกหรือไม่

บท 10

1. ระบบเสนอหลายอันให้ผู้ใช้เลือก จึงถือว่าสำเร็จถ้าผังที่ถูกอยู่ในผลที่แสดง
2. กันการเลือกค่าให้ถูกใจข้อมูลชุดเดียวกับที่รายงาน ผลบนชุด report จึงน่าเชื่อถือกว่า
3. ระบบที่ตอบทุกคำค้นจะได้ hit rate สูงแต่เดามั่ว ต้องวัดความสามารถ "ไม่เดา" ด้วย

บท 11

1. ที่ออนไลน์เป็นเวอร์ชันเก่า backend ยังเป็น NestJS และหน้าเว็บขาดหลายฟีเจอร์ จึงไม่ตรงกับโค้ดที่อธิบาย
2. ค้นหา เดินผัง ประวัติ feedback dashboard ส่วน rephrase และร่างผังใหม่ใช้ไม่ได้
3. DB นั้นใช้ร่วมกับโปรเจกต์อื่น และ runner จำตามชื่อไฟล์ ซึ่งบน branch นี้ต่างจาก `main` ผลจึงไม่ได้ทดสอบ

บท 12

1. แสดงว่าเข้าใจระบบและซื่อสัตย์ และคุมบทสนทนาได้ดีกว่าถูกจับได้ทีหลัง
2. ไม่ตรงทุกชั้น เป็นหน้าของหัวข้อ เลขหน้ารายคำสั่งอยู่ในข้อความ
3. แนะนำ: ประเมินผลก่อน เพราะยังไม่รู้ว่าการค้นหาดีแค่ไหน และผลจะบอกว่าควรปรับอะไรต่อ (ตอบตามความเห็นของตัวเองได้)

คำศัพท์ฉบับย่อ

คำ	ความหมาย
Embedding	แปลงข้อความเป็นเวกเตอร์ตัวเลข ความหมายใกล้กันเวกเตอร์ก็ใกล้กัน
Cosine similarity	วัดมุมระหว่างเวกเตอร์ เพื่อบอกความคล้าย
Threshold	เกณฑ์ขั้นต่ำ ต่ำกว่า 0.55 ไม่แสดงผล
Pre-filter	กรองตามรุ่นก่อนคำนวณคะแนน
Brute force / exact	เทียบทุกรายการ ได้ผลตรงแน่นอน แต่ช้าเมื่อข้อมูลมาก
ANN / HNSW	ค้นหาเวกเตอร์แบบประมาณ เร็วกว่าเมื่อข้อมูลมาก
RAG	ค้นข้อความแล้วให้ LLM เขียนคำตอบ
LLM	โมเดลภาษาขนาดใหญ่ที่สร้างข้อความได้
Hallucination	LLM สร้างข้อมูลที่ไม่จริงหรือไม่มีในแหล่งข้อมูล
State machine	ระบบที่มีสถานะ และเปลี่ยนสถานะตามกติกาที่กำหนด
Deterministic	ข้อมูลเข้าเดิม ได้ผลเดิมเสมอ
Pure function	ผลขึ้นกับค่าที่รับเข้า และไม่ไปเปลี่ยนสิ่งภายนอก
Transaction	หลายคำสั่งที่สำเร็จทั้งหมด หรือไม่เกิดผลเลย
Upsert	ถ้ายังไม่มีให้เพิ่ม ถ้ามีแล้วให้แก้
JWT	token ที่เซ็นแล้ว เซิร์ฟเวอร์ตรวจได้โดยไม่ต้องเก็บ session
bcrypt	hash รหัสผ่านที่ตั้งใจให้ช้า เพื่อให้เดายาก
Middleware	ฟังก์ชันที่ทำงานก่อนถึงงานหลักของ route
Migration / Seed	สคริปต์สร้างตาราง / สคริปต์ใส่ข้อมูลเริ่มต้น
Selection bias	ผลเอนเอียงเพราะตัวอย่างมาจากคนบางกลุ่ม
SUS	แบบสอบถามมาตรฐานวัดความใช้งานง่าย 10 ข้อ
Youden's J	TPR + TNR − 1 ใช้เลือก threshold ที่สมดุล